Embedded System Lab Manual

LAB EXPERIMENT NO. 16

USB Bootloader STM32 Programming with STM32F103C8T6 Blue Pill C++

Submitted To:

Sir Yasir

Submitter by:

Ahsan Basharat

Date of Submission:

13/06/2026

Introduction

In this lab, we learn about the USB Bootloader in the STM32F103C8T6 Blue Pill using C++ and STM32CubeIDE. A USB bootloader allows firmware to be uploaded to the STM32 through the USB interface without using an external programmer such as ST-Link. The bootloader initializes the USB peripheral, receives the new application firmware from a host computer, writes it into Flash memory, verifies the data, and then transfers execution to the user application.

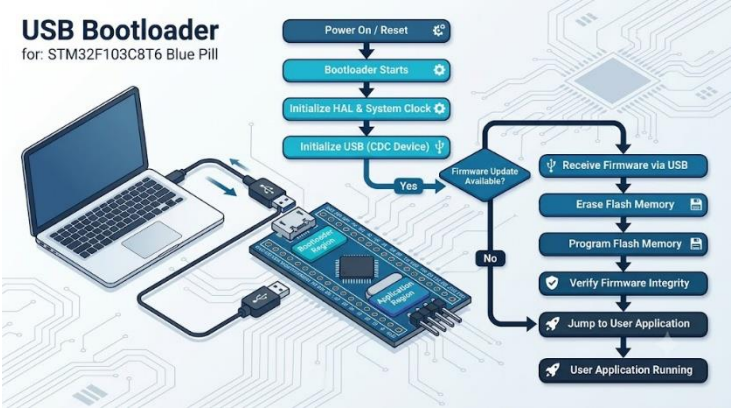
A bootloader is a small program that resides in a protected area of the microcontroller's Flash memory and executes immediately after every reset or power-up. Its primary responsibility is to prepare the system for operation and determine whether to remain in bootloader mode for a firmware update or to launch the existing user application. This makes firmware maintenance easier, especially for devices deployed in the field.

USB Bootloader

A USB Bootloader is a small program stored in the microcontroller's Flash memory that runs immediately after reset. Its primary purpose is to receive new firmware over a USB connection, program it into Flash memory, and then start the user application. Since the bootloader remains protected in Flash, the application can be updated multiple times without affecting the bootloader itself.

USB Bootloader Architecture

The USB bootloader architecture consists of a bootloader program, USB communication interface, Flash memory, and the user application. After a reset, the bootloader initializes the system clock and USB peripheral to communicate with the host computer. If a firmware update is requested, it receives the new firmware through USB, erases the required Flash memory pages, and programs the new application into Flash. The bootloader then verifies the integrity of the firmware to ensure successful programming. Finally, it relocates the vector table and



transfers execution to the user application, allowing the updated firmware to run normally.

HAL Functions Used

1. HAL_Init()

Explanation

This function initializes the Hardware Abstraction Layer (HAL) library. It configures the Flash interface, initializes the SysTick timer for delays, and prepares the STM32 peripherals for operation. It is the first HAL function called in the main() function.

2. SystemClock_Config()

Explanation

This function configures the system clock of the STM32 microcontroller. It selects the clock source (HSI or HSE), configures the PLL, and sets the CPU, AHB, and APB bus frequencies. A properly configured clock ensures stable USB communication and overall system performance.

3. MX_USB_DEVICE_Init()

Explanation

This function initializes the USB Device stack and configures the USB peripheral to operate as a USB CDC (Virtual COM Port) device. It enables communication between the STM32 bootloader and the host PC for receiving firmware updates.

4. HAL_FLASH_Unlock()

Explanation

This function unlocks the Flash memory controller, allowing the bootloader to erase Flash pages and write new firmware into the application area.

5. HAL_FLASHEx_Erase()

Syntax

FLASH_EraseInitTypeDef EraseInitStruct;

HAL_FLASHEx_Erase(&EraseInitStruct, &PageError);

This function erases one or more Flash memory pages before programming. Since Flash memory cannot overwrite existing data directly, the target pages must be erased first.

6. HAL_FLASH_Program()

Syntax

HAL_FLASH_Program(TypeProgram, Address, Data);

Example

HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD,0x08005000, 0x12345678);

Explanation

This function writes firmware data into the STM32 Flash memory at the specified address. It is used repeatedly to store the received application firmware.

Code Structure

FLASH_HAL.h

- The bootloader class inherits from the Flash class, allowing it to use Flash memory functions such as erasing, reading, and programming while adding USB bootloader functionality.
- It defines frame headers (HEAD1 = 0xAA and HEAD2 = 0xBB), parser states (head1, head2, len1, len2, type, data_read, etc.), and frame types (start_frame, flash_frame, end_frame) to identify and process firmware packets received over USB.
- The class stores important information such as the application start address, vector table pointer, jump function pointer, receive buffer, frame

length, data counters, current state, and Flash page offset, which are required during the firmware update process.

- The public functions data_in() and jump_to_app() are used to receive firmware data from the USB interface and transfer execution to the user application after the firmware has been successfully programmed.
- The private functions state_reset(), state_machine(), flash_data(), and finish() implement the bootloader's internal logic, including packet parsing, state transitions, Flash programming, and completion of the firmware update process.
- Overall, this class provides a complete USB bootloader implementation by receiving firmware packets, validating their format, storing them in Flash memory, and safely jumping to the user application once the update is complete.

```
class bootloader : public Flash
{
private:
#define HEAD1 0xAA
#define HEAD2 0xBB

public:
    enum
    {
        head1,head2,len1,len2,type,data_read,end,resetart
    };

    enum
    {
        start_frame,flash_frame,end_frame,frame_max
    };

    typedef void (*jump_app)(void);

    bootloader(uint32_t application_addr);

    void data_in(uint8_t *data, uint16_t len);
    void jump_to_app(void);

private:
    uint32_t _application_addr;
    jump_app app_jump;
    __IO uint32_t *_vectable;
    uint32_t _state;

    uint8_t _buffer[1024];
    uint16_t _buffer_cnt;
    uint16_t _data_len;
    uint16_t _data_cnt;
    uint8_t _frame_type;
    uint32_t _page_offset;

    void state_reset(void);
    void state_machine(uint8_t data);
    void flash_data(void);
    void finish(void);
};
```

USB_BOOT.cpp

bootloader::bootloader(uint32_t application_addr)

- This is the constructor of the bootloader class and is executed when a bootloader object is created.
- It initializes all internal variables, including the application start address, parser state, data counters, and Flash page offset.
- It obtains the vector table of the user application and stores the Reset Handler address as a function pointer (app_jump).
- It clears the receive buffer using memset() to ensure no old data remains before receiving firmware.

```
bootloader::bootloader(uint32_t application_addr)
    : _application_addr(application_addr),
      _state(head1),
      _buffer_cnt(0),
      _data_len(0),
      _data_cnt(0),
      _frame_type(frame_max),
      _page_offset(application_addr)
{
    _vectable = (__IO uint32_t *)_application_addr;
    app_jump = (jump_app)(*(_vectable + 1));

    memset(_buffer, 0, sizeof(_buffer));
}
```

data_in(uint8_t *data, uint16_t len)

- This function receives a block of firmware data from the USB interface.
- It processes the received data one byte at a time.
- Each byte is passed to the state_machine() function for packet parsing.
- It allows the bootloader to handle firmware packets of different sizes.

```
void bootloader::data_in(uint8_t *data, uint16_t len)
{
    for (uint32_t i = 0; i < len; i++)
    {
        state_machine(data[i]);
    }
}
```

state_reset(void)

- This function resets the bootloader parser to its initial state.
- It clears the data length, data counter, buffer counter, and frame type.
- It sets the parser state back to waiting for the first header byte (HEAD1).
- It clears the receive buffer to prepare for the next firmware frame.

```
void bootloader::state_reset(void)
{
    _state = head1;
    _data_len = 0;
    _data_cnt = 0;
    _buffer_cnt = 0;
    _frame_type = frame_max;

    memset(_buffer, 0, sizeof(_buffer));
}
```

state_machine(uint8_t data)

- This function implements the bootloader state machine that processes incoming firmware packets byte by byte.
- It verifies the frame headers, extracts the packet length, identifies the frame type, and stores the received data.
- When the buffer becomes full or an entire frame is received, it calls flash_data() to write the data into Flash memory.

```
void bootloader::state_machine(uint8_t data)
{
    switch (_state)
    {
    case head1:
        if (data != HEAD1)
        {
            state_reset();
            break;
        }

        _state = head2;
        break;

    case head2:
        if (data != HEAD2)
        {
            state_reset();
            break;
        }

        _state = len1;
        break;

    case len1:
        _data_len = ((uint16_t)data << 8);
        _state = len2;
        break;

    case len2:
        _data_len |= data;
        _state = type;
        break;

    case type:
        if (data >= frame_max)
        {
            state_reset();
            break;
```

- If the received frame is the final frame, it calls finish(); otherwise, it resets the parser to receive the next frame.

finish(void)

- This function is executed after the complete firmware has been successfully received.
- It writes a magic number (0x916614CC) into Flash memory to indicate that a valid application is available.
- It resets the STM32 using NVIC_SystemReset() so the new firmware can start executing.
- It completes the firmware update process and prepares the system for normal operation.

```
void bootloader::finish(void)
{
    uint32_t magic = 0x916614CC;

    write(0x08004C00, &magic, sizeof(magic));

    NVIC_SystemReset();

    __NOP();
}
```

flash_data(void)

- This function writes the data stored in the receive buffer into Flash memory.
- It uses the inherited write() function from the Flash class to program the application area.
- After writing, it updates the Flash address (_page_offset) to the next page.
- Finally, it clears the buffer counter so that new firmware data can be received.

```
void bootloader::flash_data(void)
{
    write(_page_offset, _buffer, _buffer_cnt);

    _page_offset += FLASH_PAGE_SIZE;

    _buffer_cnt = 0;
}
```

jump_to_app(void)

- This function transfers execution from the bootloader to the user application.
- It relocates the Vector Table Offset Register (VTOR) to the application's start address.
- It sets the Main Stack Pointer (MSP) using the application's initial stack pointer.
- Finally, it calls the application's Reset Handler (app_jump()), allowing the user program to start running.

```
void bootloader::jump_to_app(void)
{
    SCB->VTOR = _application_addr;

    __set_MSP(*_vectable);

    app_jump();
}
```

Mian.cpp

call_bl()

- This function determines whether the STM32 should remain in bootloader mode or jump to the user application after a reset or power-up.
- It enables the GPIOB clock and configures PB15 as an input with an internal pull-up resistor to detect the bootloader selection button.
- It reads the state of PB15 using HAL_GPIO_ReadPin(). If the button is pressed (logic LOW), the function returns immediately, keeping the microcontroller in bootloader mode.
- It reads the magic number (0x916614CC) from Flash memory at address 0x08004C00 to determine whether a valid application firmware has been programmed.

```
extern "C" void call_bl(void)
{
    GPIO_PinState pin_state;
    Flash flash;
    uint32_t magic = 0;

    /* Enable GPIOB clock */
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin   = GPIO_PIN_15;
    GPIO_InitStruct.Mode  = GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull  = GPIO_PULLUP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* Read boot pin */
    pin_state = HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_15);

    /* Read magic number */
    flash.read(0x08004C00, &magic, sizeof(magic));

    HAL_GPIO_DeInit(GPIOB, GPIO_PIN_15);
    __HAL_RCC_GPIOB_CLK_DISABLE();

    /* Stay in bootloader if button is pressed */
    if (pin_state != GPIO_PIN_SET)
    {
        return;
    }

    /* Stay in bootloader if magic number is invalid */
    if (magic != 0x916614CC)
    {
        return;
    }

    /* Jump to application */
    bootloader boot(0x08005000);
```

- After reading the button state and magic number, it deinitializes PB15 and disables the GPIOB clock to free hardware resources.
- If the magic number is missing or invalid, the function returns without jumping, allowing the bootloader to wait for a new firmware update.
- If the button is not pressed and the magic number is valid, it creates a bootloader object with the application start address (0x08005000) and calls jump_to_app() to start executing the user application.

main()

- A bootloader object is created, and a 2048-byte buffer is allocated to store firmware data received from the host computer over USB.

- Inside the infinite loop, the program continuously receives firmware data through USB, passes it to boot.data_in() for processing and Flash programming, echoes the received data back to the PC for confirmation, and then clears the buffer for the next packet.

```
    USB_CDC usb_cdc;
    usb_cdc.init();

    HAL_Delay(1000);

    bootloader boot(0x08005000);

    uint8_t data[2048];
    memset(data, 0, sizeof(data));

    while (1)
    {
        uint16_t len = usb_cdc.read(data, sizeof(data));

        if (len > 0)
        {
            boot.data_in(data, len);
            usb_cdc.send(data, len);

            memset(data, 0, sizeof(data));
        }
    }
}
```